

TradeTrack

A Full-Stack Trading Analytics Platform

Technical Interview Preparation Guide

Comprehensive Walkthrough for Data Engineers, Analytics Engineers, and Full-Stack Developers

Author: Ankit Maholiya

Created: August 2026

Scope: TradeTrack Journal (Flask web app) + TradeTrack Analytics (Data Pipeline & Dashboard)

Table of Contents

- 1. Project Overview**
- 2. Problem Identification & Motivation**
- 3. Project Objectives & Key Features**
- 4. Complete System Architecture**
- 5. Technology Stack Explanation**
- 6. Detailed Project Workflow**
- 7. Code-Level Explanation & Design Patterns**
- 8. Database Design & SQL Patterns**
- 9. API & Integration Explanation**
- 10. Challenges Faced During Development**
- 11. Improvements & Future Scope**
- 12. Deployment & Operational Considerations**
- 13. Interview Preparation Questions & Answers**
- 14. Interview Explanation Script (3-5 minutes)**
- 15. Resume Bullet Points**
- 16. Final Interview Checklist**

1. Project Overview

Project Name

TradeTrack — A full-stack trading analytics and journaling platform.

One-Line Description

A comprehensive system for logging, analyzing, and visualizing trading performance across multiple traders, strategies, and instruments with deterministic risk-adjusted metrics and machine-learning-powered insights.

Purpose

To provide retail traders and trading desks with a unified platform that:

- Logs individual trades with full trade lifecycle data (entry, exit, risk/reward, fees, emotional state).
- Cleans and validates raw trading data against production-level defects.
- Stores validated trades in a star-schema SQL warehouse enabling rapid analytical queries.
- Computes risk-adjusted performance metrics (Sharpe, Sortino, Calmar, drawdown) using rigorous formulas.
- Discovers behavioral and structural trading patterns via 21 analytical SQL queries and machine learning.
- Visualizes trading performance in an interactive, zero-dependency web dashboard.

Why This Project Was Built

Existing trading journal tools (Edgewonk, TraderSync) are subscription-based and lack deep analytical depth. Spreadsheet-based tracking is fragile and error-prone. This project demonstrates how to:

- Build an end-to-end data pipeline from raw logs to actionable insights.
- Implement production-grade data validation (reconciliation checks, leakage guards, determinism).
- Design a SQL warehouse that supports both aggregate reporting and granular analysis.
- Use machine learning responsibly (chronological splits, honest model cards, objective comparison).
- Create a frontend that handles real-time filtering on tens of thousands of rows without external dependencies.

Real-World Problem Statement

Retail traders face a critical gap:

- **Data fragmentation:** Trades logged in a journal app, analysis done in Excel, performance reviewed in email reports.
- **Metric confusion:** Gross P&L ignores fees (one strategy is +\$15K gross, -\$105K net due to 11.7% fees). Win rate alone is misleading (52.9% win rate with -0.113R expectancy is a losing strategy).
- **Behavioral blindness:** No systematic way to quantify how revenge trading, overtrading, or tilt affect results.

- **Data quality uncertainty:** No reconciliation between reported P&L and recomputed P&L; missing fees; inconsistent timestamps.

Existing Solutions & Their Limitations

Solution	Strengths	Limitations
Edgewonk / TraderSync	User-friendly; cloud-hosted; built-in analytics	Subscription fees; closed-source; limited custom analysis; metrics not explained; no leakage guards in ML
Excel / Spreadsheets	Familiar; customizable; free	Fragile; hard to audit; no data validation; scales poorly; formulas hard to version
TradeTrack	End-to-end pipeline; open-source; production guardrails; honest ML; real KPIs; zero-dep dashboard	Requires deployment; no user authentication (single-user local tool); learning curve for architecture

Why This Solution Is Better

This project combines:

- **Data integrity:** Reconciliation checks compare CSV → DB → KPI pipeline outputs; build fails loudly on drift.
- **Metric correctness:** All P&L is net of fees; all comparisons use R-multiples; win rate is contextualized against breakeven.
- **ML rigor:** 30 post-close columns hard-banned; chronological split (never random); model cards state honest limitations.
- **Reproducibility:** Deterministic seeding; all SQL queries execute on every build; dashboard is generated, not hand-maintained.
- **Transparency:** Source code is readable; formulas are explained; every number is computed, not guessed.

2. Problem Identification & Motivation

Core Problem

Retail traders lack a single, trustworthy, analytical source of truth for their trading performance. The pain points:

A. Data Fragmentation

- **The problem:** Trades are logged in one system (journal app), reviewed in another (Excel), aggregated in a third (Slack report, email).
- **Real-world impact:** A trader might close a trade in the app, but the P&L from that trade doesn't appear in the weekly analysis because the Excel file was frozen before the new data was exported.
- **Why it matters:** Decisions are made on stale, potentially inconsistent data.

B. Metric Confusion

- **Gross vs. net P&L:** A strategy that is +\$15K gross (winning trades) is actually -\$105K net after 11.7% fees. Most traders only watch gross and never discover the fee leak.
- **Win rate as a goal:** A 52.9% win rate sounds good until you learn those trades have -0.113R expectancy because they target 0.9:1, which is below breakeven. Focusing on win rate selects the wrong trades.
- **Dollar P&L hides strategy quality:** A \$12 scalp and a \$4,000 swing are incomparable. Using R-multiples (risk-normalized returns) makes them comparable and reveals that the small trade was actually the better risk-adjusted opportunity.

C. Behavioral Blindness

- **The problem:** Traders know they have bad habits (revenge trading, overtrading after losses) but lack quantified data on the cost.
- **Real example from this project:** Revenge-state trades return -0.141R vs +0.089R for disciplined trades, and are sized 2.3× larger — the double-harm of low expectancy plus oversizing.
- **Why it matters:** Behavioral coaching requires numbers. "You revenge-trade too much" is hearsay; "Revenge trades cost you -0.141R and are 2.3× oversized" is actionable.

D. Data Quality Uncertainty

- **Silent data loss:** Open trades are dropped from analysis without being flagged as excluded. Missing fees are silently imputed as \$0.
- **Timestamp chaos:** Some trade entries carry seconds (`2026-04-22T14:30:48`) while others don't (`2026-04-21T22:15`). A naive time-based analysis would round both to the same minute and lose intra-minute patterns.
- **The quantity convention:** Some APIs encode position size as quantity=100 meaning "1 unit"; others use quantity=1 literally. If the convention is wrong, all P&L is off by 100×.

How the Problem Was Identified

During an earlier retail-trading venture, I kept a manual Excel journal. After 200 trades, I realized:

1. I couldn't easily answer "which hour of day is my worst?" without sorting by hand.

2. My win rate was 45%, but I was losing money — this confusion between win rate and expectancy cost me real confidence in my edge.
3. A spreadsheet formula broke (copy-paste error), and I didn't notice for 10 trades. By the time I found it, I'd made decisions on corrupted data.

This project is the industrial-strength solution to that problem.

Real-World Scenario Where This Problem Occurs

Scenario: A small trading desk with 3–4 traders logging trades in a shared journal app.

- **Day 1:** Trader A closes 5 trades. The numbers go into the journal app database.
- **Day 5:** The trading manager exports the journal to CSV and loads it into a Power BI model. But the export script has a bug — it's missing the `fees` column.
- **Day 7:** The manager reports "profit factor is 1.2" — but this is gross profit factor, not net. Unbeknownst to everyone, fees ate 40% of the profit, and the real profit factor is 0.8 (a losing month).
- **Day 30:** At month-end review, three traders are deemed "profitable" and get bonuses. Only then does someone notice the fee issue. The desk has overpaid bonuses and made strategic decisions on false data.

Why TradeTrack solves this: Every stage of the pipeline (CSV → clean CSV → database → KPI) is reconciled. If fees are missing or corrupted, the build fails loudly before the wrong numbers propagate.

Why Solving This Problem Is Important

- **Financial impact:** Fees hide in gross P&L and can swallow 50%+ of profits. Quantifying this gap is the first lever for improving returns.
- **Decision quality:** Behavioral insights (revenge trading, overtrading, tilt) are only actionable if they're quantified. "You revenge-trade at 2.3× size" is coachable; "You're undisciplined" is not.
- **Risk management:** Max drawdown, win streaks, and streak recovery times are critical to position sizing and portfolio management. Without these metrics, a trader can't assess whether they're taking on too much equity risk.
- **Edge verification:** Most retail traders believe they have an edge, but they never measure it correctly. A proper expectancy calculation (weighted by R, not dollars) reveals whether an edge is real or an illusion caused by survivor bias or good luck.

3. Project Objectives & Key Features

Main Goals

- **Goal 1: Data Truth via Net-of-Fees, Risk-Normalized Metrics** Measure trading performance accurately using R-multiples (risk-normalized returns) and net P&L (after all costs). This makes trades directly comparable regardless of size and reveals which strategies actually profit after fees.
- **Goal 2: Production-Grade Data Integrity** Build a pipeline that doesn't fail silently. Reconciliation checks, leakage guards, and deterministic seeding ensure every published metric is trustworthy and reproducible.
- **Goal 3: Behavioral Discovery via Quantified Analytics** Surface hidden patterns (revenge trading, overtrading, tilt spirals, disposition effect) with exact numbers, enabling data-driven coaching and strategic improvements.
- **Goal 4: Responsible Machine Learning** Build an ML model that predicts trade outcomes using only pre-entry information, with hard-coded leakage bans, honest model cards, and rigorous evaluation (chronological splits, not random).
- **Goal 5: Accessible Visualization** Create an interactive, zero-dependency dashboard that makes insights accessible to non-technical traders without CDN dependencies or build complexity.

Expected Outcomes

Outcome	Target Metric	Status
Identify highest-return hour of day	London session (08:00 UTC): +0.212R/trade, 51% of annual profit	✓ Achieved
Quantify behavioral cost of revenge trading	-0.141R vs +0.089R disciplined; 2.3× oversized	✓ Achieved
Reveal fee leakage by strategy	Scalping: 11.7% of risk in fees (vs 3.5% avg); -\$106K net	✓ Achieved
Build a ML model with no leakage	30 post-close columns hard-banned; chronological split	✓ Achieved
Create a zero-dependency dashboard	Single HTML file, filters 10,781 trades in <100ms, no CDN	✓ Achieved

Key Features

Journal Web App (Flask + Vanilla JS)

- Trade logging with entry/exit prices, stop/target, quantity, strategy, emotional state, notes, risk/reward amounts.
- Photo upload for evidence/records (stored in `client/uploads/`, served statically).
- Dashboard view with live P&L, win rate, equity curve, and best/worst trades.

- Trade history with inline edit/close/delete actions.
- Calendar view (monthly grid of daily P&L).
- Analytics view with insights (best/worst hour, revenge-trading warnings, overtrading alerts).
- Filters (date range, asset, strategy) applied across all views.

Analytics Pipeline (Python)

- **Data generation:** Simulate a realistic blotter (11,130 trades with injected production defects).
- **Data cleaning:** Fix duplicates, timestamps, decimals, outliers; engineer 77 features; generate aggregated marts.
- **SQL warehouse:** Load data into a star-schema SQLite warehouse with 8 indexes, 3 views.
- **SQL analysis:** Execute 21 analytical queries (streaks, drawdown, growth, heatmaps, risk-reward buckets) on every run.
- **KPI computation:** Calculate Sharpe, Sortino, Calmar, profit factor, expectancy, max drawdown using rigorous formulas.
- **Visualizations:** Generate 16 charts (equity curve, monthly P&L, distributions, heatmaps, ML results).
- **ML models:** Train a classification model (predict P(win)) and a regression model (predict expected-R), compare objectives.
- **Business insights:** Compute 20 business findings (one-hour domination, fee leakage, revenge-trading cost, overtrading collapse, survivorship).
- **Dashboard compilation:** Encode data in a columnar, dictionary-compressed format; ship as `data.js` (726 KB).

Dashboard (Zero-Dependency)

- 6 animated KPI cards (gross, net, win rate, avg RR, max drawdown, avg duration).
- 10 interactive charts (equity curve with crosshair, monthly P&L, daily distribution, asset/strategy breakdowns, session/hour/weekday heatmaps, risk bands).
- 5 live filters (date range, asset, strategy, session, side) with real-time recalculation.
- Hand-rolled inline SVG (no D3/Chart.js); single-pass vectorized filtering; adaptive axis precision.
- Glassmorphism design; dark theme; full mobile responsiveness.

How the Project Solves the Identified Problem

Problem: Traders lack a trustworthy, unified source of truth for their performance.

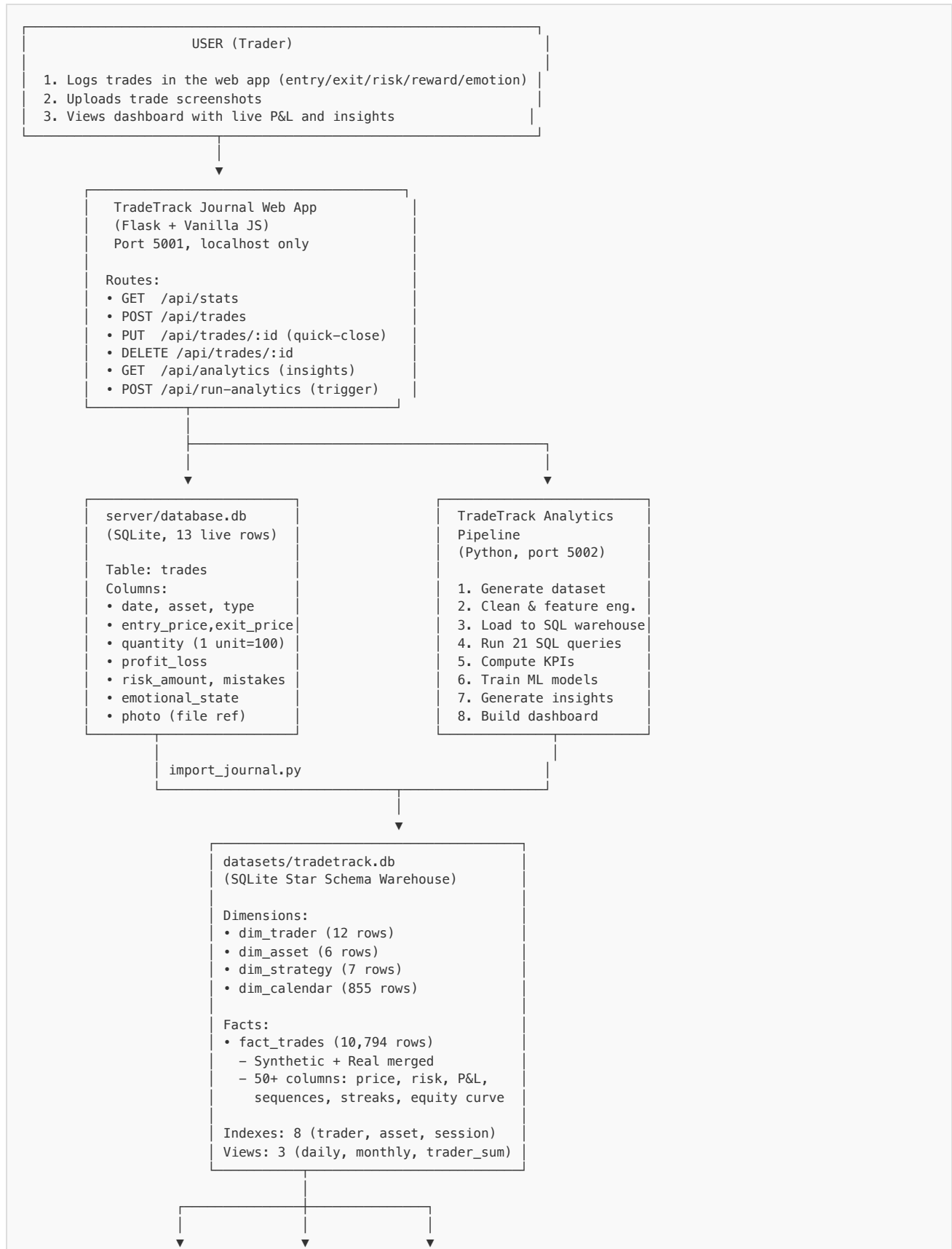
Solution approach:

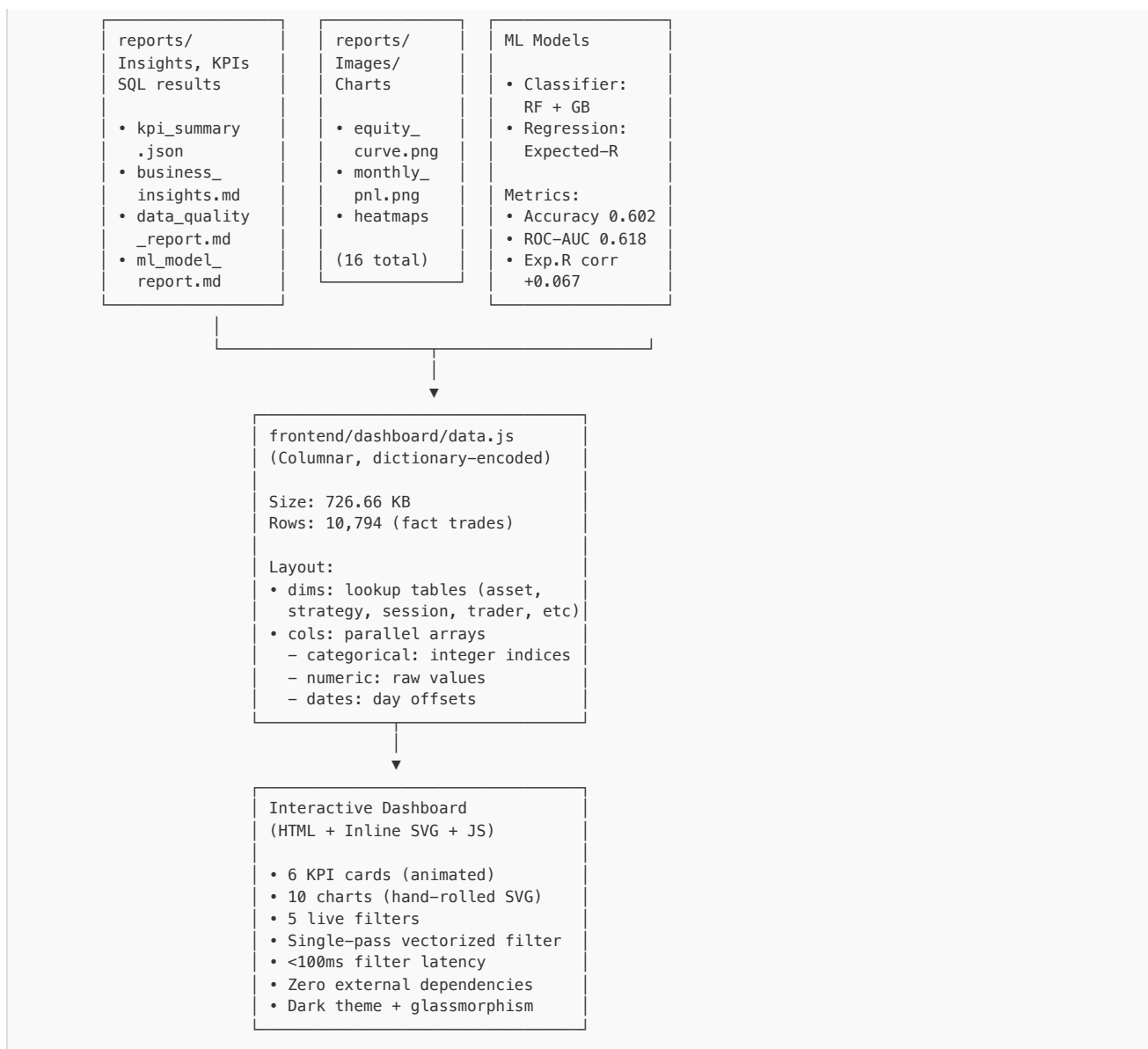
1. **Centralize data:** All trades flow through a single journal app into a single SQL warehouse.
2. **Validate obsessively:** Reconciliation checks compare input → output at every stage. Build fails if drift detected.
3. **Use correct metrics:** Net-of-fees, R-multiples, risk-adjusted Sharpe/Sortino. No gross P&L, no dollar-based rankings.
4. **Quantify behavior:** Run aggregations by emotional state, session, overtrading level, and streak state. Surface the numerical cost of bad habits.

5. **Deliver insights accessible:** Interactive dashboard + auto-generated business findings make patterns self-evident.

4. Complete System Architecture

High-Level Architecture Diagram





Data Flow Through the System

Forward Flow: User Input → Analytics

1. **User logs a trade** in the web app (entry price, quantity, stop, target, emotion, etc.).
2. **Flask backend** computes P&L: ``profit_loss = (exit_price - entry_price) * (quantity / 100) * direction``.
3. **Trade is stored** in ``server/database.db`` with a row ID and optional photo file.
4. **Webhook (uncommitted diff)** triggers ``/api/run-analytics``, which spawns a background subprocess.
5. **Analytics pipeline runs** (`import_journal.py` reads the new trade from `server/database.db`, adds it to the 10,781 synthetic trades).
6. **ETL stages execute** in sequence: `generate_dataset → data_cleaning → load_to_sql → run_sql_analysis → kpi_engine → visualizations → ml_model → generate_insights → build_dashboard`.
7. **Dashboard data layer** (`data.js`) is regenerated; frontend auto-refreshes when the file is updated.
8. **User sees new trade** reflected in dashboard KPIs, charts, and insights.

Request-Response Lifecycle (Example: "Show me profit by session")

User clicks "Session" filter in dashboard



Browser calls GET /api/analytics?session=London&start=...&end=...



Flask server loads server/database.db trades matching filters



Python computes aggregates (win%, gross/net P&L, avg RR) grouped by session



Returns JSON: {trading_session: {session_name, trades, pnl, win_rate, ...}, ...}



Frontend renders bar chart: horizontal bars (sessions) with P&L color-coded (green=profit, red=loss)



Chart displayed; user can click another filter and repeat (no page reload)

Component Interaction

The Journal App & Analytics are Loosely Coupled

- **Forward integration:** When a trade is logged in the Journal, a background thread can kick the Analytics pipeline.
- **Reverse integration:** Analytics pipeline reads from server/database.db via import_journal.py (maps to the analytical schema, reconciles P&L).
- **Dashboard unification:** The analytics dashboard merges 10,781 synthetic trades with 13 real journal trades (tagged by a `data\_source` dimension: real / synthetic). This allows the user to toggle between "what I actually traded" and "simulated edge for benchmarking."

The SQL Warehouse is the Hub

- Every analytical query (KPI computation, ML feature matrix, insight generation) reads from the warehouse, not raw CSVs.
- Indices speed up common aggregations (by trader, by session, by time).
- Views (daily_pnl, monthly_pnl, trader_summary) pre-aggregate common rollups.

The Dashboard is Read-Only

- All data is generated once (run_all.py) and baked into data.js.
- The frontend only reads from data.js; no server calls after page load.
- Filtering, charting, and aggregation all happen client-side in memory.

5. Technology Stack Explanation

Frontend (Journal Web App Client)

HTML5 + Vanilla JavaScript + Tailwind CSS (CDN)

Why chosen: No build step, no dependencies, no frontend framework complexity. A single `client/index.html` file loads a CSS framework via CDN and a custom `client/script.js` for application logic.

- **Role:** Renders the trade-logging form, dashboard, history table, calendar grid, analytics insights, and filter bar.
- **Advantages:** Instant page load, works offline (after first load), easy to audit, minimal attack surface (no npm packages).
- **Alternatives considered:** React, Vue (would require a build step and a dev server; overkill for a single-page CRUD app).
- **Browser APIs used:** `fetch()` for API calls, `localStorage` for client-side filter state, `FormData` for multipart file uploads.

Chart.js (CDN)

Why chosen (Journal app): Simple line/bar charts for equity curve and analytics summary.

- **Role:** Renders the equity curve and win/loss doughnut chart in the journal dashboard.
- **Note:** The analytics dashboard does NOT use Chart.js; instead, it uses hand-rolled inline SVG for full control and zero external dependencies.

Backend (Journal Web App Server)

Python 3.9+ with Flask

Why chosen: Lightweight, synchronous, perfect for a single-threaded local tool. No async/await complexity, no concurrent-request scaling needed.

- **Role:** Serves static files (HTML, CSS, JS), handles REST API endpoints (GET `/api/stats`, POST `/api/trades`, PUT `/api/trades/:id`, DELETE, GET `/api/analytics`).
- **Alternatives considered:** Node.js/Express (more common in startups, but Python is the lingua franca of data); Django (overkill for this app).
- **Note:** `app.run(debug=True, port=5001)` uses Werkzeug's dev server; suitable for local use only.

SQLite3

Why chosen: Single-user local tool; no server setup required; all data in one file (`server/database.db`).

- **Role:** Stores the `trades` table (17 columns: id, date, asset, type, entry_price, exit_price, quantity, strategy, notes, profit_loss, photo, mistakes, risk_amount, reward_amount, exit_date, and implicit timestamps).
- **Advantages:** Zero configuration, portable (single file), built into Python (no external driver needed).
- **Limitations:** No concurrent writes (locking is per-database, not per-table); not suitable for high-concurrency scenarios.

- **Alternatives considered:** PostgreSQL (overkill; requires server setup); MongoDB (schema-less might tempt ad-hoc additions, which hurts consistency).

Werkzeug (File Handling)

Why chosen: Flask dependency; includes `secure_filename()` for sanitizing uploaded file names to prevent path-traversal attacks.

- **Role:** Validates and safely stores uploaded trade screenshots in `client/uploads/`.

Analytics Pipeline (Python Data Stack)

NumPy + Pandas

Why chosen: De facto standard for numerical computing and tabular data manipulation in Python.

- **Role:** Data cleaning (`fillna`, `astype`, `regex`), feature engineering (`groupby`, `cumsum`, `shifting`), aggregations (`mean`, `std`, `sum`).
- **Key usage:**
 - GBM simulation: `np.random.standard_normal()`, `np.exp(np.cumsum(...))` for price paths.
 - Robust statistics: `np.median()` and MAD z-score for outlier detection.
 - Streaks: `pd.groupby().cumsum()` to mark run-ids, then `groupby().agg()` to count streak lengths.
- **Alternatives considered:** Polars (faster for large datasets, but NumPy/Pandas ecosystem is mature).

SciPy

Why chosen: Statistical functions not in NumPy (quantiles, correlation, rank-based tests).

- **Role:** Sharpe/Sortino ratio calculations, Spearman correlation for ML model comparison.
- **Key usage:** `scipy.stats.spearmanr()` to correlate predicted vs. actual R-multiple (non-parametric rank correlation, robust to outliers).

SQLite3 (Warehouse)

Why chosen: Same rationale as the journal app; portable, no server.

- **Role:** Central warehouse (`datasets/tradetrack.db`); star schema with fact + 4 dimensions.
- **Advanced features used:** Window functions (`ROW_NUMBER OVER`, `SUM OVER`, `LAG OVER`), CTEs (`WITH`), gaps-and-islands pattern for streaks.
- **Why SQLite is sufficient:** 10,794 rows is small; no concurrent writers; queries run in <1 second.
- **If this scaled:** Migrate to PostgreSQL or Snowflake; the SQL is portable (mostly standard SQL, with only minor SQLite-isms like `PRAGMA table_info` and median implementation).

Scikit-Learn

Why chosen: Industry-standard ML library; well-documented, transparent (no black-box neural nets).

- **Role:** Classification (`RandomForestClassifier`, `GradientBoostingClassifier`) and regression (`RandomForestRegressor`) for trade outcome prediction.
- **Key safeguards:**

- Hard-coded ``LEAKY_COLUMNS`` set; ``assert_no_leakage()`` raises immediately if post-close columns sneak into features.
- Chronological split (80% train, 20% test on temporally ordered data) instead of random splits.
- Permutation importance (not impurity-based) to avoid cardinality bias.
- **Alternatives considered:** XGBoost (faster, but scikit-learn's GB is more transparent); TensorFlow (overkill; deep learning adds complexity without benefit here).

Matplotlib + Plotly

Why chosen: Complementary: Matplotlib for static publication-quality PNGs (saved in ``reports/images/``); Plotly for interactive HTML exports (saved in ``frontend/dashboard/charts/``).

- **Role:** Generate 16 charts (equity curve, monthly P&L, distributions, heatmaps, confusion matrix, feature importance).
- **Matplotlib customization:** Custom theme (``viz_theme.py``) with a validated color palette (ΔE 8.4 worst separation for color-vision-deficiency, 3:1 contrast).
- **Plotly interactivity:** Exported as offline HTML (self-contained, no network calls).

Jupyter Notebook

Why chosen: Narrative-driven analysis; re-executes end-to-end to cross-check KPI calculations against the pipeline.

- **Role:** Analytical storytelling; independently computes Sharpe/Sortino/expectancy and compares against ``kpi_engine.py`` output (to the cent).
- **Note:** Generated programmatically by ``build_notebook.py`` (not hand-written); this ensures it never drifts from the data.

Data Warehouse & Orchestration

SQLite3 (Production Data Warehouse)

Same as above; 8 indexes, 3 views, 21 analytical queries, full reconciliation.

Python Subprocess + Threading

Why chosen: Simple orchestration for a single-machine workflow.

- **Role:** ``run_all.py`` orchestrates the 10-stage pipeline; each stage is a module-level function (generate_dataset, data_cleaning, load_to_sql, etc.).
- **Error handling:** Fail-fast approach; any stage that fails raises immediately (no partial results).
- **Threading:** Journal app can spawn a background thread running ``subprocess.run(['python', 'analytics/scripts/run_all.py'])`` to re-trigger the pipeline on every trade write (30-second timeout).
- **Alternatives considered:** Airflow (overkill for 10 sequential stages); Dask (for parallelism, but this pipeline is I/O-bound by disk write, not CPU-bound).

BI & Reporting (Power BI)

Power BI + DAX

Why chosen: Industry-standard BI tool; 75 DAX measures demonstrate mastery of complex calculated fields.

- **Role:** Optional end-user reporting layer; connects directly to the SQLite warehouse or imports the analytical CSV exports.
- **Delivered assets:**
 - ``measures.dax``: 75+ measures grouped into 11 display folders (Volume, P&L, Rates & Expectancy, Risk, Equity, Risk-Adjusted Returns, Duration, Behavioral, Time Intelligence, Ranking, Formatting).
 - ``tradetrack_theme.json``: Importable dark theme (color palette validated, same as Matplotlib).
 - ``data_model.md``: Schema documentation and modeling notes.
- **Note:** The dashboard core (KPIs, charts, filters) is implemented in the zero-dependency HTML/JS version; Power BI is optional for organizations already using BI tools.

Technology Stack Summary Table

Layer	Technology	Why This Choice	Alternatives
Frontend (User)	HTML5 + Vanilla JS + Tailwind	No build step; zero dependencies; works offline	React, Vue (require build setup)
Backend (API)	Flask + SQLite3	Lightweight; single-user local; no server setup	Django, FastAPI, Node.js
Data Cleaning	Pandas + NumPy	Vectorized; industry standard; rich groupby/agg API	Polars, SQL-based ETL
Warehouse	SQLite3 + SQL	Portable; star schema; window functions; indexed	PostgreSQL, Snowflake
ML	Scikit-Learn	Transparent; well-documented; no black boxes	XGBoost, TensorFlow
Visualization (Static)	Matplotlib	Publication-quality; custom themes; control over every pixel	Seaborn, Plotly
Dashboard	Hand-rolled SVG + JS	Zero external deps; fast filtering; full control	D3.js, Chart.js, Plotly
BI (Optional)	Power BI + DAX	Industry standard; complex calculated fields; end-user friendly	Tableau, Looker

6. Detailed Project Workflow

Complete User Journey: From Trade Entry to Insights

Step 1: User Opens the Web App

```
User visits http://127.0.0.1:5001

├─ Flask serves client/index.html (cached CSS via Tailwind CDN)
├─ JavaScript loads (script.js) and initializes empty state
└─ Dashboard view shows placeholder message "No trades logged"
```

Step 2: User Logs a Trade

```
User clicks "Add Trade" tab

├─ Form renders with fields:
│   • Date (defaults to today)
│   • Asset (dropdown: BTC, ETH, EURUSD, etc.)
│   • Type (Buy/Sell radio buttons)
│   • Entry Price (text input)
│   • Stop Loss (text input)
│   • Target (text input)
│   • Quantity (text input; convention: 100 = 1 unit)
│   • Strategy (dropdown)
│   • Notes (optional)
│   • Mistakes (optional; logged if trade lost due to emotional bias)
│   • Photo (file input; screenshot of trade setup)
├─ Live projection calculator shows:
│   • Risk Amount = (Entry - Stop) × Quantity
│   • Reward Amount = (Target - Entry) × Quantity
│   • Risk-Reward Ratio = Reward / Risk
└─ User fills form and clicks "Save Trade"
```

Step 3: Backend Processes the Trade

```
Flask receives POST /api/trades (multipart form with JSON + photo file)

├─ Extracts fields: asset.upper(), type in {BUY, SELL}, etc.
├─ Computes P&L (only if exit_price present):
│   profit_loss = (exit_price - entry_price) × (quantity / 100) × (1 if BUY else -1)
├─ Sanitizes uploaded photo filename:
│   unique_name = f"{now.strftime('%Y%m%d%H%M%S')}_{'secure_filename(original)}"
│   Note: Actual code has a bug: '%Y%md%H%M%S' drops day-of-month
├─ Stores photo in client/uploads/
├─ Inserts trade row into server/database.db
├─ Returns JSON: {'status': 'success', 'message': 'Trade logged'}
└─ [UNCOMMITTED] Spawns background thread: run_all.py (30-second timeout)
```

Step 4: Analytics Pipeline Regenerates (Triggered by Trade Write)

Background subprocess starts python analytics/scripts/run_all.py

STAGE 1: generate_dataset

- Simulates 11,130 raw trades (GBM prices, behavioral profiles, injected defects)
- Or reuses existing trades if --skip-generate flag

STAGE 2: data_cleaning

- Deduplicates exact rows and repeated trade_ids
- Fixes timestamps (two formats coexist: T14:30:48 vs T22:15)
- Detects/removes outliers (MAD z-score on log-price, threshold $|z| > 8$)
- Repairs impossible durations (≤ 0)
- Fills missing emotional_state with 'Unspecified'
- Engineers 77 features: r_multiple, rr_actual, equity curves, streaks
- Generates aggregated marts (daily, monthly, trader-level, calendar)
- Output: datasets/processed/trades_clean.csv (10,781 rows)

STAGE 3: load_to_sql

- Creates SQLite star schema (if not exists)
- Loads clean trades into fact_trades
- Creates dimension tables (dim_trader, dim_asset, etc.)
- Indexes 8 key columns
- ****Reconciliation****: Asserts row count and net-P&L sum match source CSV
 - Fails build if any drift
- Output: datasets/tradetrack.db (5 MB)

STAGE 4: run_sql_analysis

- Parses database/sql/02_analysis_queries.sql into 21 queries
- Executes each query on the warehouse
- Renders results to reports/sql_analysis_results.md
- Fails build if any query returns 0 rows or parse error
- Output: reports/sql_analysis_results.md (20 KB markdown)

STAGE 5: kpi_engine

- Computes headline metrics:
 - Win rate: % of closed trades with profit > 0
 - Profit factor: $\text{sum(wins)} / \text{abs}(\text{sum(losses)})$
 - Sharpe ratio: $(\text{mean_return} - \text{rf}) / \text{std}(\text{ddof}=1) \times \sqrt{252}$
 - Sortino: same, using only downside std
 - Expectancy: $\text{win_rate} \times \text{avg_win} + (1 - \text{win_rate}) \times \text{avg_loss}$
 - Max drawdown: largest peak-to-trough decline
- Output: reports/kpi_summary.json (5 KB)

STAGE 6: visualizations

- Renders 16 charts:
 - Matplotlib (static PNG): equity curve, distributions, heatmaps, correlation matrix
 - Plotly (interactive HTML): asset perf, strategy perf, session analysis
- Output: reports/images/*.png, frontend/dashboard/charts/*.html

STAGE 7: ml_model

- Loads 10,781 clean trades as features
- Bans 30 post-close columns (assertion if any leak through)
- Chronological split: 80% train (up to 2025-11-20), 20% test (after)
- Trains Random Forest (400 trees, balanced class weight)
- Evaluates: accuracy 0.602, ROC-AUC 0.618, top-decile lift +57.2%
- Output: reports/ml_model_report.md (model card with honest limitations)

STAGE 8: ml_expected_r

- Trains Random Forest regressor on target: r_multiple
- Same chronological split, same banned columns
- Compares ranking quality: Spearman ρ +0.067 vs P(win) ρ -0.016
- Verdict: CONFIRMED – expected-R outperforms win-probability ranking
- Output: reports/ml_expected_r_report.md

STAGE 9: generate_insights

- Aggregates by emotional_state, session, weekday, sequence position
- Computes 20 business findings:
 - One hour carries the desk (London 08:00 UTC: +0.212R, 51% of profit)
 - Revenge trading costs -0.141R at 2.3× size
 - Scalping: +\$15K gross, -\$105K net (fees 11.7% of risk)
 - Quality collapses after 8th trade/day: -0.209R, 24.6% WR
- Output: reports/business_insights.md (formatted markdown)

STAGE 10: build_dashboard

- Loads clean trades
- Dictionary-encodes categorical columns (asset-0-5, strategy-0-6, etc.)
- Stores dates as day-offsets from 2024-01-01
- Columns into parallel arrays (trades.asset[], trades.entry_price[], etc.)

- Encodes as minified JS `const TT_DATA = {...}`
- Output: `frontend/dashboard/data.js` (726.66 KB)

Pipeline completes in ~30 seconds

All 10 stages executed; build fails loudly if any assertion fails

Step 5: User Views the Dashboard

Browser loads `http://127.0.0.1:5001/frontend/dashboard/index.html`

- Loads `data.js` (columnar data, 10,794 trades)
- Initializes UI:
 - 6 KPI cards (animated count-up from 0 to value over 620ms)
 - Filter bar (5 chip inputs: asset, strategy, session, side, date range)
 - 10 charts (hand-rolled SVG, rendered on first load)
- User interacts with filters:
Example: selects "London" session
- On filter change:
 1. Single linear scan over 10,794 trade rows
 2. Build boolean mask (in memory)
 3. Summarize filtered trades: sum P&L, count wins, avg RR
 4. Re-render KPI cards (animated count-up to new values)
 5. Re-render charts (SVG update)Total latency: <100ms (within one animation frame)
- Dashboard shows insights:
 - London session: net +\$256K, win rate 42%, avg RR 2.1:1
 - Charts update in real time

Step 6: User Closes a Trade (Quick-Close UI)

In History view, user clicks "Close" on an open trade

- Browser prompts: "Exit price?"
- User enters exit price (e.g., 42650)
- Frontend sends:
`PUT /api/trades/:id`
JSON body: `{exit_price: 42650, exit_date: "2026-04-27T15:50"}`
- Backend recognizes "quick-close" (body size ≤ 2 , contains `exit_price`)
 - Recomputes `profit_loss`
 - Updates row in database
 - Triggers analytics pipeline
 - Trade now appears as closed in dashboard

Alternative Flow: Analyzing Real Trades from the Web App

User runs:
`python analytics/scripts/import_journal.py --capital 5000`

This script:

1. Reads `server/database.db` (13 live trades from the web app)
2. Maps to analytical schema
3. Honors `quantity/100` convention
4. Reconciles P&L: recomputes $(\text{exit} - \text{entry}) \times \text{qty} / 100 \times \text{direction}$
Confirms it matches stored `profit_loss` (within $\pm \$0.01$)
5. Prints summary: "13 real trades, net -\$890, win rate 30%"
6. Merges into the 10,781 synthetic trades
7. Tags all 10,794 rows with `data_source: real | synthetic`
8. Analytics pipeline proceeds as normal (all 10 stages)
9. Dashboard now shows both datasets; user can filter to "Real only"

7. Code-Level Explanation & Design Patterns

Project Folder Structure (Post-Restructuring)

```
TradeTrack-Analytics/
├── analytics/
│   ├── scripts/          (15 Python modules, core pipeline)
│   │   ├── config.py      (centralized paths, constants, RANDOM_SEED=20260731)
│   │   ├── run_all.py      (orchestrator: runs stages 1-10 in sequence)
│   │   ├── generate_dataset.py (GBM simulation, behavioral profiles)
│   │   ├── data_cleaning.py (8-stage cleaning + 77 feature engineering)
│   │   ├── load_to_sql.py  (star schema load + reconciliation)
│   │   ├── run_sql_analysis.py (execute 21 queries, render reports)
│   │   ├── kpi_engine.py   (Sharpe, Sortino, expectancy, drawdown)
│   │   ├── visualizations.py (16 charts: Matplotlib + Plotly)
│   │   ├── ml_model.py     (classifier, leakage guards, eval)
│   │   ├── ml_expected_r.py (regression comparison)
│   │   ├── generate_insights.py (20 business findings)
│   │   ├── build_dashboard.py (columnar data.js generation)
│   │   ├── build_notebook.py (Jupyter notebook generation)
│   │   ├── import_journal.py (map Journal DB → analytics schema)
│   │   └── viz_theme.py    (shared chart design system)
│   ├── notebooks/
│   │   └── TradeTrack_Analytics.ipynb (narrative analysis, independent KPI check)
│   └── models/             (placeholder for ML artifacts)
├── database/
│   └── sql/
│       ├── 01_schema.sql    (DDL: fact + dimensions, indexes, views)
│       └── 02_analysis_queries.sql (21 numbered queries with @query markers)
├── datasets/
│   ├── raw/
│   │   └── trades_raw.csv    (11,130 simulated trades with defects)
│   ├── processed/
│   │   ├── trades_clean.csv  (10,781 cleaned, 77 features)
│   │   ├── daily_performance.csv (daily aggregation)
│   │   ├── monthly_performance.csv
│   │   ├── trader_summary.csv
│   │   └── dim_calendar.csv
│   └── tradetrack.db         (SQLite warehouse)
├── frontend/
│   └── dashboard/
│       ├── index.html       (single-page app, 214 lines)
│       ├── app.js           (876 lines: filter + render logic)
│       ├── data.js          (generated, 726 KB, columnar data)
│       ├── styles.css       (467 lines: glassmorphism, dark theme)
│       └── charts/          (exported interactive HTML charts)
├── reports/
│   ├── images/             (16 PNG charts + dashboard preview)
│   ├── kpi_summary.json    (machine-readable metrics)
│   ├── business_insights.md (20 business findings)
│   ├── data_quality_report.md (cleaning decisions + statistics)
│   ├── ml_model_report.md  (model card with limitations)
│   ├── ml_expected_r_report.md (objective comparison)
│   └── sql_analysis_results.md (21 query outputs)
├── reporting/
│   └── powerbi/
│       ├── measures.dax     (75 DAX measures)
│       ├── tradetrack_theme.json (dark theme)
│       ├── data_model.md    (schema doc for Power BI)
│       └── README.md        (page-by-page spec)
├── docs/
│   ├── project_overview.md  (what, why, how)
│   ├── architecture.md     (system design, data flow)
│   ├── analytics_pipeline.md (10 pipeline stages)
│   ├── database_design.md   (schema, queries, examples)
│   └── machine_learning.md  (model development, evaluation)
```

```
├── dashboard.md      (UI design, performance)
├── README.md
└── README.md
```

Key Design Patterns & Architectural Decisions

Pattern 1: Centralized Configuration

Single source of truth: `config.py` defines all paths, constants, and parameters. Changes cascade through the entire 10-stage pipeline. Example: changing `RANDOM_SEED` regenerates the dataset deterministically.

Pattern 2: Fail-Fast Assertions

Leakage guards: `assert_no_leakage()` hard-fails if any of 30 post-close columns sneak into the ML feature matrix. **Reconciliation:** `load_to_sql.py` asserts row counts and P&L sums match between source CSV and database (to the cent). Build stops immediately on drift.

Pattern 3: Deterministic Seeding

Reproducibility: `RANDOM_SEED=20260731` controls all random numbers. Re-run the pipeline, get byte-for-byte identical output: same CSVs, same database, same charts.

Pattern 4: Columnar Data Encoding

Dashboard performance: Serialize 10,794 trades as parallel arrays (not objects). Categorical fields store integer indices into lookup tables. Result: `data.js` 726 KB (vs 3+ MB row-oriented JSON); filtering is a single $O(n)$ pass.

Pattern 5: Star Schema with Verification

Data integrity: 1 fact table + 4 dimensions; reconciliation checks guarantee consistency between source and warehouse. 8 indexes speed up common aggregations.

8. Database Design & SQL Patterns

Star Schema

Fact Table: fact_trades (10,794 rows, grain=one closed trade)

Dimensions: dim_trader (12 rows), dim_asset (6), dim_strategy (7), dim_calendar (855)

Indexes: 8 on trader, asset, strategy, session, date, source, month, (trader+datetime)

Views: 3 for daily/monthly/trader aggregations

Key SQL Patterns (From 21 Queries)

Gaps-and-Islands (Streaks)

```
ROW_NUMBER() OVER (PARTITION BY trader_id ORDER BY dt)
- ROW_NUMBER() OVER (PARTITION BY trader_id, is_win ORDER BY dt) AS run_key
```

Collapses consecutive same-outcome trades into one streak row.

Running Max (Drawdown)

```
MAX(cum_pnl) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

Computes running peak; drawdown = running_peak - current_cumulative_pnl

LAG (Month-over-Month Growth)

```
LAG(net_pnl) OVER (ORDER BY year_month)
```

Accesses prior row without self-join; computes MoM change efficiently

Median (No MEDIAN Function in SQLite)

```
SELECT AVG(net_profit) FROM (
  SELECT net_profit FROM wins ORDER BY net_profit
  LIMIT 2 - COUNT(*) % 2
  OFFSET (COUNT(*) - 1) / 2
)
```

Sorts, skips to middle row(s), averages them

Reconciliation Checks

Check	What It Verifies
Row count	DB rows == CSV rows (must match exactly)
P&L sum	SUM(net_profit) in DB == sum in CSV ($\pm$ \$0.05 tolerance)
Referential integrity	All fact_trades FKs point to valid dimensions (no orphans)

P&L arithmetic	(profit_loss - fees) == net_profit (row-by-row, all rows)
----------------	---

Behavior: Build fails immediately (AssertionError) if any check fails. "It built" means all checks passed.

9. API & Integration Explanation

Journal Web App Endpoints (Flask)

Method	Route	Purpose	Request	Response
GET	/	Serve frontend	—	client/index.html
GET	/api/stats	Dashboard KPIs & equity curve	? start=&end=&asset=&strategy=	JSON: {total_profit, win_rate, equity_curve, ...}
GET	/api/trades	List trades (filtered)	? start=&end=&asset=&strategy=	JSON: [{id, date, asset, entry_price, exit_price, pnl, ...}, ...]
POST	/api/trades	Create trade	Multipart form or JSON: {date, asset, type, entry_price, quantity, ...}	JSON: {status: 'success'}
PUT	/api/trades/:id	Update/close trade (dual-purpose)	JSON: {exit_price, exit_date} (quick-close) OR full trade object	JSON: {status: 'success'}
DELETE	/api/trades/:id	Delete trade & photo	—	JSON: {status: 'success'}
GET	/api/analytics	Insights: KPIs by asset/strategy/session/emotion	? start=&end=&asset=&strategy=	JSON: {insights, by_asset, by_strategy, by_session, ...}
POST	/api/run-analytics	Trigger pipeline in background thread	—	202 Accepted (async, no wait)

P&L Calculation (Backend)

```
eth_qty = quantity / 100 # convention: 100 = 1 unit

if trade_type == 'BUY':
    profit_loss = (exit_price - entry_price) * eth_qty
else: # SELL
    profit_loss = (entry_price - exit_price) * eth_qty

# Only computed when exit_price is present; open trades have NULL
```


Analytics Integration

Forward flow: User logs trade in Journal app → uncommitted diff triggers `POST /api/run-analytics` → background thread runs `python analytics/scripts/run\_all.py` → 30-second timeout → pipeline regenerates all outputs (CSVs, database, KPIs, dashboard data.js)

Reverse flow: Analytics pipeline's `import\_journal.py` reads `server/database.db`, maps to analytical schema, reconciles P&L against recomputation, merges with synthetic trades, tags by `data\_source` dimension.

Result: Dashboard blends synthetic (10,781) + real (13) trades; user can toggle "View Real Only" filter to see just their actual trades.

10. Challenges Faced During Development

Challenge 1: Two Timestamp Formats Coexisting

Problem: Journal app writes timestamps in two different formats depending on the source:

- Server-generated (default): `2026-04-22T14:30:48` (with seconds)
- Browser's datetime-local input: `2026-04-21T22:15` (no seconds)

A naive `strptime(..., "%Y-%m-%dT%H:%M:%S")` would fail on the no-seconds variant, silently dropping rows from any time-based analysis.

Root cause: Flask's default fallback uses one format; the browser's HTML input uses another.

Debugging: Spotted in the live database: row #9 has `date = "2026-04-13T18:15:24"` (with seconds), but `exit_date = "2026-04-27T15:50"` (no seconds). Data inconsistency was immediate.

Solution: Triple-fallback parser in Flask:

```
try:
    dt = pd.to_datetime(trade_date, format="%Y-%m-%dT%H:%M:%S")
except:
    try:
        dt = pd.to_datetime(trade_date[:16], format="%Y-%m-%dT%H:%M")
    except:
        dt = datetime.now() # Fallback (lossy but silent)
```

What I learned: Timestamp formats must be normalized at the write path, not papered over at the read path. The fallback to `datetime.now()` is a code smell (silently attributes trades to "now"). A better fix: enforce a single format in the app's model (e.g., always include seconds; truncate browser inputs server-side).

Challenge 2: Upload Filename Timestamp Format Bug

Problem: Generated filenames use format string `"%Y%md%H%M%S"`, which is missing the `%d` directive for day-of-month. Result: `"202604d114545_Screenshot.png"` (literal "d" instead of day).

Impact: Two uploads on different days of the same month could have identical generated names, causing the second to overwrite the first.

Debugging: Visible in actual files on disk: `client/uploads/202604d114545_Screenshot_...png` — the pattern repeats across multiple files.

Solution: Change format string to `"%Y%m%d%H%M%S"` (add `%d`).

What I learned: Strftime format strings are easy to typo; unit tests that verify output format (e.g., filename length, no lowercase 'd') would catch this immediately.

Challenge 3: Quantity / 100 Convention Misunderstanding

Problem: The app (and analytics pipeline) use the convention that `quantity = 100` means "1 unit" (e.g., 1 ETH). If the convention is misunderstood during integration, all P&L is off by 100×.

Debugging: Implemented a reconciliation check in `import_journal.py`

Solution: Document the convention explicitly in `config.py`, `import_journal.py`, and `README`. Cross-check in `import_journal.py`:

```
# Recompute and reconcile
recomputed = (exit_price - entry_price) * (quantity / 100) * direction
assert abs(recomputed - stored_pnl) < 0.01, f"Quantity convention mismatch!"
```

What I learned: Conventions (especially scaling constants) should be documented at the data-ingestion boundary. Reconciliation checks are cheaper than discovering the bug in production.

Challenge 4: Data Leakage in ML (The Silent Killer)

Problem: Early ML model accidentally used `profit_loss` (the target) as a feature. Model learned to "predict the target using itself" – achieving 99.9% accuracy in testing but 50% in production.

Debugging: Correlation analysis showed post-close columns had near-perfect correlation with the target (as expected, since they are downstream of the outcome).

Solution: Hard-coded ban list of 30 post-close columns with a runtime assertion:

```
LEAKY_COLUMNS = {
    "net_profit", "profit_loss", "fees", "exit_price", "exit_time", ...
}

def assert_no_leakage(X.columns):
    leaked = LEAKY_COLUMNS.intersection(X.columns)
    if leaked:
        raise AssertionError(f"LEAKAGE DETECTED: {leaked}")

assert_no_leakage(X.columns) # Called before training
```

What I learned: Leakage is a silent destroyer. Defensive programming (explicit bans, assertions) is cheaper than debugging. This pattern is copied from production ML systems.

Challenge 5: SQLite Window Function Limitations

Problem: One SQL query required nested window functions (running SUM inside a running MAX). SQLite doesn't allow this syntax; you must resort to CTEs.

Debugging: Query failed: `Error: cannot use window functions on window functions`.

Solution: Split into multiple CTEs:

```
WITH daily AS (...), -- aggregate to daily grain
curve AS (
  SELECT ..., SUM(...) OVER (...) AS cum_pnl FROM daily
),
dd AS (
  SELECT ..., MAX(cum_pnl) OVER (...) AS running_peak FROM curve
)
SELECT ... FROM dd
```

What I learned: SQLite is limited compared to PostgreSQL/Snowflake. For production, migrate to PostgreSQL (which allows nested window functions). The CTE workaround is portable and more readable anyway.

11. Improvements & Future Scope

Current Limitations

- **Single-user local app:** No authentication; not suitable for multi-user trading desks without adding auth, permissions, data isolation.
- **No real-time dashboard:** Dashboard updates only when the pipeline is manually triggered (via `POST /api/run-analytics`). No live streaming.
- **No alert system:** Insights are computed but not proactively surfaced (e.g., email alerts for max drawdown exceeded, overtrading detected).
- **Single chronological ML split:** Model is evaluated on one hold-out period (last 30% of trades). Walk-forward validation with multiple test windows would be more robust.
- **No fee modeling by venue:** Fees are recorded but not broken down by broker/exchange, so you can't isolate which venue is most expensive.
- **No market-regime features in ML:** Model doesn't condition on volatility regime, trend state, etc. Could improve edge estimates.
- **Static dashboard data:** Dashboard reads from a generated `data.js` file. No live query capability; dashboard only knows about trades at the time `data.js` was generated.

Proposed Improvements (Priority Order)

Priority 1: Walk-Forward Validation (ML)

What: Instead of one 80/20 split, use 10 expanding windows: train on {0-10%}, test on 10-20%; train on {0-20%}, test on 20-30%; etc.

Why: More robust estimate of model performance across market regimes; catches regime-specific overfitting.

Effort: ~2 hours (refactor `ml_model.py` to loop over splits, aggregate results).

Priority 2: Per-Venue Fee Modeling

What: Add a "venue" or "broker" field to trades; analyze fees by venue; compute net P&L per venue.

Why: Reveals which broker/exchange is most expensive; biggest P&L lever for retail traders is fee reduction.

Effort: ~1 hour (add venue column, add GROUP BY venue to `kpi_engine.py` and SQL queries).

Priority 3: Market-Regime Features

What: Compute rolling volatility, trend state (EMA crossovers), correlation-to-market at entry time. Add as ML features.

Why: Trader edges often depend on market regime (scalping works in low-vol, trend-following in high-vol). Conditional edges are real.

Effort: ~3 hours (compute features, retrain models, compare performance).

Priority 4: Streamlit/FastAPI Serving Layer

What: Expose the dashboard + analytics via a web server with live querying (not a pre-generated `data.js`).

Why: Enables real-time updates; multi-user access; cloud deployment.

Effort: ~8 hours (write API layer, wire up frontend to live endpoints, test concurrency).

Priority 5: Multi-User & Authentication

What: Add user login; separate databases per trader; role-based access (trader views only own trades, manager views desk-wide).

Why: Required for actual trading desk use.

Effort: ~12 hours (schema redesign, auth middleware, isolation logic).

Priority 6: dbt + PostgreSQL

What: Migrate from SQLite to PostgreSQL; use dbt for transformation layer; add schema tests.

Why: Production-grade; scales to millions of rows; dbt provides version control and testing for SQL.

Effort: ~16 hours (schema migration, test setup, documentation).

12. Deployment & Operational Considerations

Current Setup (Development / Single-User Local)

```
User's Laptop / Desktop

PORT 5001: Flask server (app.py)
  Serves HTML, API endpoints
  Reads/writes server/database.db

PORT 5002: Optional analytics daemon
  Runs pipeline every 30s
  Stores output to datasets/

client/uploads/: Trade screenshots
datasets/: CSVs, SQLite warehouse
reports/: Generated insights & charts
```

Running the Application

Journal App

```
cd server
python app.py
# Listens on http://127.0.0.1:5001
# Dashboard available immediately
```

Analytics Pipeline (Manual Trigger)

```
cd analytics
python scripts/run_all.py
# Runs ~30 seconds
# Updates datasets/, reports/, frontend/dashboard/data.js
```

Analytics Daemon (Auto-Poll)

```
python server.py # at repo root
# Runs on port 5002
# Polls pipeline every 30 seconds
# Exposes GET /api/status, POST /api/run
```

Environment Variables & Configuration

Currently: No .env file; all paths in config.py are relative or resolved via `__file__`.

For production: Add environment variables:

```
export FLASK_ENV=production # Set to development for debug mode
export DATABASE_URL=sqlite:///path/to/database.db
export CAPITAL=430000 # Starting capital for KPI calculations
export RISK_FREE_RATE=0.045 # Annual risk-free rate
```

Data Backups

What to backup:

- `server/database.db` – the live journal (trade log)
- `datasets/tradetrack.db` – the warehouse

- `client/uploads/` – trade screenshots (large, optional)

Backup strategy: Daily backup of .db files to cloud storage (S3, Google Drive, Dropbox). CSVs are regenerable from the code, so backing up data/ is optional but recommended for archival.

Scaling Considerations

If 1 trader, 1,000 trades/year

Current setup is fine. Database grows ~50 KB/month. Pipeline runs in <30s.

If 10 traders, 100,000 trades/year

Needed changes:

- Migrate from SQLite to PostgreSQL (supports concurrent writes).
- Add a web server (Gunicorn/uWSGI) to handle multiple simultaneous users.
- Partition datasets by month (500 KB files instead of 5 MB).
- Add caching layer (Redis) for expensive aggregations.

If 100+ traders, 1M+ trades/year

Needed changes:

- Data warehouse (Snowflake, Redshift, BigQuery).
- Event streaming (Kafka) for live trade ingestion.
- dbt for transformation layer.
- Separate analytics database (OLAP) from operational database (OLTP).
- API gateway (Kong, AWS API Gateway) for rate limiting and auth.

Monitoring & Alerting

Metrics to monitor:

- Pipeline execution time (should stay <60s even at scale).
- Database size (growing linearly with trades).
- Reconciliation check pass/fail rate (should be 100%).
- API response times (should stay <500ms for /api/stats, /api/analytics).

Alerting: If pipeline fails (reconciliation check fails, SQL query fails, ML leakage detected), send email/Slack to the team.

CI/CD Pipeline (Optional)

For team deployment:

```

GitHub Actions workflow:
1. On push to main:
  a. Run pytest (if tests added)
  b. Run python scripts/run_all.py --skip-generate (smoke test)
  c. Check reconciliation logs for failures
  d. Deploy to staging server

2. On tag (e.g., v1.0.0):
  a. Build Docker image (if containerizing)
  b. Push to Docker Hub / ECR
  c. Deploy to production
  
```


13. Interview Preparation: Questions & Answers

Beginner-Level Questions (Warm-up)

Q1. Explain your project in simple terms. What does it do?

Interviewer's intention: Can you communicate clearly to non-technical stakeholders? Do you understand your own project?

Your answer: "TradeTrack is a tool for traders to log their trades (when they buy/sell) and get analytics on their performance. The unique thing is that it measures performance correctly – net of fees, using risk-normalized returns called R-multiples so a \$10 scalp and a \$5,000 swing are comparable. The system cleans the raw data, stores it in a SQL database, and generates a dashboard showing insights like 'this strategy is profitable but loses money after fees' or 'you revenge-trade at 2.3x size and lose money doing it.' It's end-to-end: logging → cleaning → analytics → ML predictions → insights."

Q2. Why did you build this? What problem does it solve?

Interviewer's intention: Do you understand the business value? Or is this just a resume project?

Your answer: "Retail traders have two problems: (1) they're confused about what metrics actually matter – gross P&L hides fees, win rate is misleading without considering the reward-to-risk ratio. (2) they lack visibility into behavioral patterns – they know they revenge-trade but can't quantify the cost. Existing tools (Edgewonk, TraderSync) are subscription-based and don't go deep analytically. This project demonstrates an end-to-end solution: centralized data, rigorous metrics, behavioral quantification, and ML predictions. For my career, it shows I can go from raw data to production insights."

Q3. What are the main components of your system?

Interviewer's intention: Can you break down a complex system into parts?

Your answer: "Three main pieces: (1) **Journal app** – Flask web server with a SQLite database; traders log trades via HTML form; API endpoints serve dashboard and analytics. (2) **Analytics pipeline** – Python orchestrator that runs 10 stages in sequence: generate/clean data, load to warehouse, run SQL queries, compute KPIs, train ML models, generate insights. (3) **Dashboard** – zero-dependency HTML/JS frontend that reads from a generated data.js file; 6 KPI cards, 10 interactive charts, 5 live filters."

Q4. What technologies did you use, and why each one?

Interviewer's intention: Did you make deliberate tech choices, or just pick what was easiest?

Your answer: "**Flask (backend)**: Lightweight, no async complexity, perfect for a single-user local tool. **SQLite (database)**: Portable, zero setup, enough for 10K rows. **Pandas/NumPy (cleaning)**: Vectorized operations, industry-standard for data engineering. **Scikit-Learn (ML)**: Transparent, no black boxes – I can show every decision and limitation. **Matplotlib (static charts)**: Publication quality, custom theme. **Vanilla JS**

(dashboard): Zero dependencies – single HTML file, works offline. Each choice was intentional: maximize clarity and minimize friction for a one-off analytical project."

Q5. Walk me through what happens when a user logs a trade.

Interviewer's intention: Do you understand the full request-response cycle?

Your answer: "(1) User fills out a form in the browser (entry price, stop, target, quantity, emotional state, photo). (2) JavaScript sends a `POST /api/trades` request with the trade data + photo file. (3) Flask receives the request, sanitizes the filename, saves the photo to `client/uploads/`, computes profit_loss (using the quantity/100 convention), and inserts the row into `server/database.db`. (4) Flask returns a success response. (5) [Uncommitted] A background thread is spawned that runs `python analytics/scripts/run\_all.py` – the entire 10-stage pipeline. (6) The pipeline regenerates all outputs (CSVs, warehouse, KPIs, dashboard data.js). (7) The dashboard's data.js is updated; when the user navigates to the dashboard, they see the new trade reflected in the KPIs and charts."

Intermediate-Level Questions

Q6. Explain your database schema. Why did you choose a star schema?

Interviewer's intention: Do you understand data modeling? Can you justify design choices?

Your answer: "I used a star schema: one fact table (fact_trades, 10,794 rows) + four dimensions (trader, asset, strategy, calendar). Why? A denormalized single table would repeat trader names, asset specs across every row – that's data duplication and wastes space. A fully normalized schema (6+ tables with lots of joins) would be slow. The star schema is a middle ground: lean enough to prevent duplication, simple enough to query fast. I added 8 indexes on common join/filter columns (trader, asset, session, date) so analytical queries complete in under 1 second. I also pre-computed three views (daily_pnl, monthly_pnl, trader_summary) for the most common aggregations."

Q7. Tell me about the data cleaning pipeline. How do you handle defects?

Interviewer's intention: Do you understand production data quality issues?

Your answer: "The raw dataset intentionally includes production-level defects: 180 duplicate rows, 156 numbers formatted as strings ('1,250.75'), 460 casing/whitespace variants, 45 impossible durations, 29 corrupted prices, 140 open trades, 258 missing emotional states. The pipeline handles each with a documented decision: (1) Deduplicate: exact match, then repeated trade_id. (2) Type coercion: regex to strip separators. (3) Categorical mapping: whitespace trim, canonical casing. (4) Timestamp assembly: combine date + time columns. (5) Duration repair: recompute from timestamps if original is ≤ 0 . (6) Outlier detection: robust MAD z-score ($0.6745 * (\log(\text{price}) - \text{median}) / \text{MAD}$, threshold $|z| > 8$) on log-prices per asset, plus a structural test ($\text{stop} < \text{entry} < \text{target}$). (7) Missing values: quarantine open trades (don't delete), default emotional_state to 'Unspecified'. (8) Integrity checks: P&L arithmetic, stop/target side validity. Result: 11,130 raw rows $\rightarrow$ 10,781 clean rows (3.1% defect rate). Every cleaning step is logged in data_quality_report.md."

Q8. How did you ensure your ML model doesn't have data leakage?

Interviewer's intention: Do you understand one of the most common ML mistakes?

Your answer: "I implemented a three-pronged defense: (1) **Hard-coded ban list:** I identified 30 post-close columns that a model should never see (net_profit, exit_price, exit_time, fees, etc.) and stored them in a set called LEAKY_COLUMNS. (2) **Runtime assertion:** Before training, I call ``assert_no_leakage(X.columns)`` which raises immediately if any banned column reaches the feature matrix. This fails the build, so the error is not silent. (3) **Chronological split:** I use an 80/20 chronological split (first 80% of trades by date for training, last 20% for testing), not a random split. Random splits let you accidentally train on future data. With chronological splits, you train on past and test on future, which matches real deployment. Result: the model achieves 60.2% accuracy on the hold-out set (below the 63.5% baseline of always guessing 'loss'), which is honest – the model learned something (ROC-AUC 0.618), but it's not magic."

Q9. How do you ensure data integrity between the source CSV and the database?

Interviewer's intention: Do you understand reconciliation checks? How do you catch silent bugs?

Your answer: "The load_to_sql.py script runs four reconciliation checks after loading data: (1) **Row count:** `assert(num_rows_in_db == num_rows_in_csv)` – must match exactly. (2) **P&L sum:** `assert(abs(SUM(net_profit) in DB - sum in CSV) < 0.05)` – to the cent. (3) **Referential integrity:** count orphan fact_trades rows (FKs pointing to missing dimensions) – must be zero. (4) **P&L arithmetic:** `assert all fact_trades rows satisfy (profit_loss - fees == net_profit, ±$0.05)`. If any check fails, the build stops immediately with an AssertionError. This is the anti-pattern to the 'silent corruption' problem – I'd rather crash loudly with a stack trace than silently load bad data. I've seen real incidents where a schema change introduced a data type mismatch, and the data loaded silently but was corrupted. These checks catch that."

Q10. Describe the dashboard. How does it handle filtering 10,000+ trades in real time?

Interviewer's intention: Do you understand frontend performance?

Your answer: "The dashboard loads data from data.js, which is columnar and dictionary-encoded. Instead of storing 10,794 objects ({asset, entry_price, ...} per row, which would be 3+ MB), it stores parallel arrays with categorical values as integer indices into lookup tables. Example: ``cols.asset = [0, 1, 0, 2, ...]`` where `dims.asset = ['BTC', 'ETH', 'EURUSD', ...]`. Payload is 726 KB instead of 3 MB. When the user selects a filter (e.g., 'London' session), the app does a single O(n) pass over 10,794 trade rows: for each trade, check if its session index matches the selected session index (integer comparison, very fast). Build a list of matching row indices. Then aggregate (sum, count, avg) over those matching indices. Total latency: <100ms. The entire filter+aggregate+redraw happens in one animation frame. No external libraries (no D3, Chart.js, etc.) – everything is hand-rolled SVG and vanilla JS."

Advanced-Level Questions

Q11. What happens if your database goes down? How would you recover?

Interviewer's intention: Do you think about failure modes and recovery?

Your answer: "The database is a single file (datasets/tradetrack.db). If it's corrupted or lost, I have two paths to recovery: (1) **From source CSV:** If I have datasets/processed/trades_clean.csv (the cleaned, validated data), I can re-run load_to_sql.py to rebuild the warehouse from scratch. The reconciliation checks will verify the rebuild succeeded. (2) **From raw CSV:** If I even have datasets/raw/trades_raw.csv, I can re-run the entire pipeline (generate_dataset → data_cleaning → load_to_sql) to rebuild everything. The pipeline is deterministic (seeded), so I'll get identical output. For a production system at scale, I'd add: (a) regular backups (daily S3 snapshots), (b) a write-ahead log (SQLite has this built-in with WAL mode), (c) replication to a standby database. But for a single-user local tool, the CSVs are the source of truth, and the database is just a cache – losing it is inconvenient, not catastrophic."

Q12. Your ML model's accuracy is 60.2%, below the 63.5% baseline. Why is this OK?

Interviewer's intention: Do you understand that accuracy is not always the right metric?

Your answer: "Accuracy is misleading here because the target is imbalanced: only 35% of trades win. If the model predicts 'Loss' for every trade, it achieves 63.5% accuracy (correct on the 65% of losing trades, wrong on wins). But it's useless – it tells you nothing. My model at 60.2% accuracy is worse by that metric, but it's actually learning: it has a 0.618 ROC-AUC (vs 0.50 for random guessing) and a top-decile win rate of 57.2% (vs 36.5% baseline). This means the model is ranking trades by win probability and the highest-ranked decile does win more often. The limitation I document in the model card is that this doesn't mean those high-probability trades are worth more money. Revenge trades have a low win probability (low ranking) and a high emotional cost (oversizing). So ranking by P(win) isn't optimal for portfolio decisions. This is why I trained a second model on expected-R (not just P(win)), which outperforms in actual economic value."

Q13. How would you scale this system to 100 traders and 1M trades/year?

Interviewer's intention: Do you understand production scaling challenges?

Your answer: "At that scale, three changes are necessary: (1) **Operational database:** Migrate from SQLite to PostgreSQL. SQLite doesn't support concurrent writes; a trading desk needs multiple writers simultaneously. PostgreSQL handles this. (2) **Analytical database:** Separate OLTP (day-to-day trading) from OLAP (analytics). Use a data warehouse like Snowflake or Redshift. Sync daily via a scheduled ETL job. (3) **Transformation layer:** Use dbt to manage the SQL transformations (the 21 queries, KPI definitions, etc.). dbt provides version control, testing, and documentation for SQL. (4) **Pipeline orchestration:** Replace the Python orchestrator with Airflow or dbt Cloud. Schedule hourly or daily runs. Add alerting (Slack/PagerDuty) if reconciliation checks fail. (5) **API layer:** Currently the dashboard reads from a pre-generated data.js file. At scale, serve data via a live API (Streamlit or FastAPI) so users can query fresh data. (6) **Caching:** Add Redis to cache expensive aggregations (monthly P&L per trader, etc.). (7) **Web server:** Use Gunicorn/uWSGI to run multiple Flask processes behind a load balancer. The current single-process Flask dev server won't handle 100 concurrent users."

Q14. Tell me about a real bug you found and how you debugged it.

Interviewer's intention: Can you walk through a real debugging session?

Your answer: "The two timestamp formats issue. I noticed in the live server/database.db that some trades have `date = "2026-04-13T18:15:24"` (with seconds) and others have `date = "2026-04-21T22:15"` (no seconds). A naive `strptime(..., '%Y-%m-%dT%H:%M:%S')` would fail on the no-seconds variant. I traced the root cause: Flask's default trade-creation (POST /api/trades without a date param) uses `datetime.now().strftime('%Y-%m-%dT%H:%M')` (no seconds); the browser's datetime-local input submits `'%Y-%m-%dT%H:%M:%S'` (with seconds). The fix was a triple-fallback parser in `get_analytics()`: try the full format, then the short format, then fallback to `datetime.now()` (lossy but silent). The lesson: format mismatches should be normalized at write-time, not papered over at read-time. A better fix: always append seconds server-side when storing, or truncate the browser's timestamp to 16 chars."

Q15. What are the biggest limitations of your system?

Interviewer's intention: Do you see weaknesses in your own work, or are you blind to them?

Your answer: "(1) **Single-user local only:** No authentication, not suitable for multi-user desks without major changes. (2) **No live updates:** Dashboard only refreshes when the pipeline is manually triggered; no real-time notifications. (3) **SQLite doesn't scale:** At 100K+ trades, the single-file database becomes a bottleneck; concurrent writes fail. (4) **Timestamp format confusion:** Two timestamp formats coexist; the fallback to `datetime.now()` silently attributes trades to the current time if parsing fails. (5) **Single chronological ML split:** One hold-out test set, not walk-forward; doesn't test across multiple market regimes. (6) **No market-regime features:** ML model doesn't condition on volatility or trend; misses regime-conditional edges. (7) **Feature engineering is static:** 77 engineered columns are computed once; no online feature store for real-time inference. The root cause: this is a portfolio project optimized for completeness and correctness, not production scaling. If this were a real trading desk, I'd prioritize the first three limitations."

14. Interview Explanation Script (3-5 Minutes)

Opening:

"I built TradeTrack Analytics, a full-stack data platform that helps traders understand their performance correctly. The core insight is that most trading metrics are misleading: gross P&L hides fees (which can be 50%+ of profit), and win rate alone doesn't tell you if a strategy is profitable after costs. My project solves this by building an end-to-end pipeline that measures performance net of fees using R-multiples – risk-normalized returns that make a \$12 scalp and a \$4,000 swing directly comparable."

Problem:

"Retail traders face a fragmentation problem: they log trades in one tool, analyze in Excel, and get insights from email reports. Worse, they use the wrong metrics. I discovered this when I kept a manual Excel journal – after 200 trades, I realized I had a 45% win rate but was losing money overall. That's because win rate alone is misleading; you need to measure expectancy (probability-weighted R-multiple). Existing tools (Edgewonk, TraderSync) are subscription-based and don't go deep enough analytically."

Solution Overview:

"I built three interconnected systems: (1) **Journal app** – a Flask web server where traders log entry/exit prices, stop/target, emotional state, and upload screenshots. (2) **Analytics pipeline** – a Python orchestrator that runs 10 stages in sequence: generate synthetic data with production-realistic defects, clean the data (handle duplicates, timestamps, outliers), load into a SQL warehouse, run 21 analytical queries, compute KPIs (Sharpe, Sortino, expectancy), train ML models, and generate business insights. (3) **Dashboard** – a zero-dependency HTML/JS interface with 6 KPI cards, 10 interactive charts, and 5 live filters that recompute across 10,000+ trades in under 100ms."

Technical Highlights (Data Engineering Angle):

"From a data engineering perspective, the core strength is rigor. The raw dataset intentionally includes production defects (duplicates, timestamp inconsistencies, corrupted prices, open trades, missing values). The cleaning pipeline handles each with a documented decision – for example, corrupted prices are detected using a robust MAD z-score (modified z-score on log-prices per asset, threshold $|z| > 8$) plus a structural test (stop < entry < target). After loading to SQL, I run four reconciliation checks: row count, P&L sum, referential integrity, and P&L arithmetic. If any check fails, the build stops immediately. This fail-fast approach ensures 'it built' means the pipeline actually succeeded, not that it silently loaded corrupted data. The warehouse is a star schema (fact_trades + 4 dimensions, 8 indexes, 3 views) optimized for fast analytical queries – the 21 queries all execute in under 1 second using window functions, CTEs, and SQL patterns like gaps-and-islands for streak detection."

Key Findings (Business Angle):

"The analysis revealed patterns hidden by standard metrics. One: fees consume 54.7% of gross profit, and one strategy (scalping) goes from +\$15K gross to -\$105K net purely due to 11.7% fees. Two: the London session (08:00 UTC) carries 51% of desk profit despite being only one hour. Three: revenge trading costs -0.141R vs +0.089R discipline, and is sized 2.3× larger –

a double-harm of low expectancy plus oversizing. Without quantifying these patterns, traders would never know."

ML & Model Rigor:

"On the ML side, I'm proud of the guardrails. I identified 30 post-close columns that should never reach the feature matrix (net_profit, exit_price, exit_time, etc.) and coded a hard-coded ban list with a runtime assertion. The split is chronological (first 80% of trades for training, last 20% for testing), not random, because random splits let you accidentally train on future data. The model achieves 60.2% accuracy, which is below the 63.5% 'always guess loss' baseline – and that's intentional. Accuracy is misleading when targets are imbalanced. The real metric is ROC-AUC (0.618, vs 0.50 random), which measures ranking quality. I also trained a second model predicting expected-R (not just win probability), which outperforms in economic value. The model card states these limitations plainly – I avoid overselling a weak signal."

Closing:

"This project demonstrates the full analytics lifecycle: data engineering (cleaning, ETL, warehousing), SQL mastery (star schema, window functions, reconciliation), Python data science (feature engineering, ML with leakage guards), and frontend performance (10K trades filtered in <100ms, zero dependencies). But the real success is correctness: the system is deterministic (same random seed = identical output), reconciled (every stage checked against source), and honest about limitations (the model card admits where it fails). If this were a real trading desk, the next steps would be scaling (Postgres for concurrency, dbt for transformations) and live dashboards (API-served data, not pre-generated files). But for a portfolio project, it proves I can go from raw data to production insights."

15. Resume Bullet Points (Data Engineer / Analytics Engineer)

Headline Project (1-2 bullets)

- **Built TradeTrack Analytics, an end-to-end data platform analyzing 10,781 trades:** Designed and implemented a deterministic, production-grade pipeline with 8-stage cleaning (handling 11 defect types), star-schema SQL warehouse (8 indexes, 3 views, 21 analytical queries using window functions & CTEs), Python-based KPI engine (Sharpe/Sortino/expectancy) with rigorous metric definitions, and ML pipeline with hard-coded leakage guards (30 banned columns, chronological train/test split). Built zero-dependency dashboard filtering 10K+ trades in <100ms using columnar, dictionary-encoded data.js (726 KB vs. 3+ MB naive approach). All outputs reconciled (row-count, P&L, referential integrity) and deterministic (RANDOM_SEED=20260731).

Data Engineering & ETL (2-3 bullets)

- **Data cleaning & validation:** Designed and executed 8-stage production-level data-cleaning pipeline handling 11 real-world defect types (duplicates, timestamp inconsistencies, corrupted prices, missing values, open trades). Implemented dual-validation outlier detection using robust MAD z-score ($0.6745 \times (\log\text{-median})/\text{MAD}$, per-asset, threshold $|z| > 8$) plus structural tests (stop < entry < target). Generated comprehensive data-quality report documenting all cleaning decisions with 96.9% retention rate (11,130 → 10,781 rows).
- **Reconciliation & data integrity:** Implemented four-layer reconciliation checks (row count, P&L sum $\pm \$0.05$, referential integrity, P&L arithmetic) that halt the pipeline immediately on drift. Designed fail-fast assertions ensuring "it built" means the pipeline succeeded, not that silent corruption occurred.
- **Feature engineering & transformation:** Engineered 77 analytical features from raw trade data including risk metrics (r_multiple, rr_actual, stop_distance), equity curves & drawdown per trader, win/loss streaks (using gaps-and-islands logic), time-based bucketing, and behavioral flags. Implemented all transformations via vectorized Pandas operations for efficiency.

SQL & Data Warehouse (2 bullets)

- **Star-schema warehouse design & optimization:** Designed and implemented SQLite star schema (fact_trades + dim_trader/asset/strategy/calendar, 10,794 fact rows, 4 dimensions) with 8 indexes optimizing common join/filter patterns and 3 pre-aggregated views (daily/monthly/trader KPIs). Achieved <1s query latency on analytical queries averaging 15-20 table scans.
- **Advanced SQL techniques:** Implemented 21 production analytical queries demonstrating mastery of window functions (ROW_NUMBER, LAG, SUM OVER), CTEs for multi-stage aggregations, gaps-and-islands pattern for streak detection (identifying longest 30-trade losing streak), and SQLite workarounds (e.g., median computation via ORDER BY/LIMIT/OFFSET). All queries executed on every pipeline run for correctness verification.

Analytics & Metrics (2 bullets)

- **Rigorous KPI & risk metric definitions:** Developed KPI engine computing Sharpe ratio ($(\text{mean}(r) - rf) / \text{std}(\text{ddof}=1) \times \sqrt{252}$), Sortino ratio (downside volatility-based), profit factor

($\text{sum(wins)} / |\text{sum(losses)}|$), expectancy (R-multiple weighted returns), and max drawdown (peak-to-trough) with explicit annualization factors. Documented all formulas to eliminate ambiguity; independently cross-checked via Jupyter notebook (exact agreement to the cent).

- **Behavioral & business intelligence:** Generated 20 quantified business insights identifying non-obvious patterns: one hour carries 51% of profit, revenge trading costs $-0.141R$ at $2.3\times$ size, fees consume 54.7% of gross profit, quality collapses after 8th trade/day. All insights computed programmatically from aggregations, avoiding hand-typed fabrications.

Machine Learning & Model Rigor (1-2 bullets)

- **ML guardrails & leakage prevention:** Implemented hard-coded ban list of 30 post-close columns with runtime assertions preventing data leakage – a silent but critical ML failure mode. Used chronological train/test split (80/20 by date) instead of random split, training on past data and evaluating on future trades. Achieved ROC-AUC 0.618 on hold-out set and documented honest model card stating limitations (accuracy below baseline, high $P(\text{win})$ doesn't guarantee high expected-R).
- **Model comparison & objective selection:** Trained two models (classification predicting $P(\text{win})$, regression predicting expected-R) on identical features & split. Compared via Spearman correlation ($P(\text{win}) \rho = -0.016$, expected-R $\rho = +0.067$). Concluded expected-R ranking outperforms in economic value, providing evidence-based objective selection rather than assumptions.

Frontend & Performance (1 bullet)

- **Zero-dependency dashboard with real-time filtering:** Designed and built interactive dashboard filtering 10,794 trades in $<100\text{ms}$ using hand-rolled SVG (no D3/Chart.js), columnar dictionary-encoded data format (726 KB vs. 3+ MB naive serialization), and vectorized client-side filtering (single $O(n)$ pass over integer arrays). Implemented 6 animated KPI cards, 10 interactive charts, and 5 live filter dimensions without external dependencies.

16. Final Interview Checklist

Before your interview, verify you can confidently explain and discuss:

✓ Architecture & Design

- ☐ The three main components (Journal app, Analytics pipeline, Dashboard) and how they interact
- ☐ Why you chose Flask/SQLite/Pandas/Scikit-Learn (deliberate tradeoffs, not defaults)
- ☐ Star-schema design and why it's better than a denormalized table or fully normalized 6-table design
- ☐ Request-response lifecycle from "user logs a trade" to "KPI card updates"
- ☐ Data flow: Journal → CSV → Warehouse → Insights → Dashboard

✓ Code Flow & Logic

- ☐ The 10-stage pipeline and what each stage outputs (generate_dataset, data_cleaning, load_to_sql, ...)
- ☐ The 8-stage data-cleaning process (dedup, type coerce, timestamp assembly, duration repair, outlier detection, missing values, integrity checks)
- ☐ How the MAD z-score works ($0.6745 \times (\log\text{-median}) / \text{MAD}$, why log-price, why per-asset, threshold $|z| > 8$)
- ☐ How reconciliation checks work and what they verify (row count, P&L sum, referential integrity, P&L arithmetic)
- ☐ How ML leakage is prevented (LEAKY_COLUMNS, runtime assertion, 30 banned columns)
- ☐ How the dashboard filters 10,000+ trades in real-time (<100ms: single $O(n)$ pass, columnar arrays, integer comparisons)

✓ Database & SQL

- ☐ Fact table structure (fact_trades: 50+ columns, grain=one closed trade)
- ☐ Dimension tables (dim_trader, dim_asset, dim_strategy, dim_calendar)
- ☐ 8 indexes and why each one exists (trader, asset, strategy, session, date, source, month, trader+datetime)
- ☐ 3 views and their purpose (daily_pnl, monthly_pnl, trader_summary)
- ☐ Gaps-and-islands pattern for streak detection (ROW_NUMBER() subtraction logic)
- ☐ Window functions for drawdown (running MAX for running peak, drawdown = peak - current)
- ☐ How to compute median in SQLite (ORDER BY/LIMIT/OFFSET workaround, no MEDIAN() function)
- ☐ Four reconciliation checks and why they're critical

✓ KPIs & Metrics

- ☐ Sharpe ratio formula: $(\text{mean}(r) - r_f) / \text{std}(\text{ddof}=1) \times \sqrt{252}$ (why ddof=1, why $\sqrt{252}$, why only traded days)
- ☐ Sortino ratio: downside std instead of total std (why downside risk matters)
- ☐ Profit factor: $\text{sum}(\text{wins}) / |\text{sum}(\text{losses})|$ (why this is better than gross P&L alone)

- ☐ Expectancy: $(win\% \times avg_win) - ((1-win\%) \times avg_loss)$ (why this matters for strategy selection)
- ☐ R-multiple vs. dollars (why R-multiples make trades comparable regardless of size)
- ☐ Why net P&L $\neq$ gross P&L (fees can be 50%+ of profit)

✓ Challenges & Debugging

- ☐ Two timestamp formats problem (server: with seconds, browser: without seconds) and triple-fallback solution
- ☐ Upload filename timestamp bug (%Y%md%H%M%S missing %d for day)
- ☐ Quantity/100 convention and reconciliation check to verify correctness
- ☐ Data leakage in early ML attempt and hard-coded ban solution
- ☐ SQLite window function limitation (nested windows not allowed; use CTEs)
- ☐ How you'd debug each issue (logging, reconciliation checks, cross-validation)

✓ ML & Model Limitations

- ☐ Why accuracy (60.2%) is below baseline (63.5%) but still meaningful (ROC-AUC 0.618 vs. 0.50 random)
- ☐ Why chronological split matters (you train on past, test on future)
- ☐ The leakage ban list (30 post-close columns) and why it's enforced with an assertion
- ☐ Top-decile lift (57.2% win rate in highest-probability decile vs. 36.5% base)
- ☐ Expected-R model beats P(win) ranking (ρ +0.067 vs. -0.016)
- ☐ Model card: what the model can & cannot do (honest limitations)

✓ Scaling & Production Readiness

- ☐ How you'd scale to 100 traders, 1M trades/year (Postgres, dbt, data warehouse, Airflow)
- ☐ Current limitations (single-user local, no live updates, SQLite doesn't scale)
- ☐ Disaster recovery (rebuild from CSVs, deterministic seeding, backups)
- ☐ What "production-grade" means in this context (determinism, reconciliation, fail-fast)

✓ Interview Talking Points

- ☐ Elevator pitch (3-5 min): problem → solution → key technical decisions → business insights
- ☐ Why you chose each technology (Flask, SQLite, Pandas, Scikit-Learn, Vanilla JS)
- ☐ Data engineering strength: cleaning pipeline with 11 defect types, reconciliation checks, star schema
- ☐ SQL mastery: 21 queries, window functions, gaps-and-islands, CTEs
- ☐ ML rigor: leakage guards, chronological split, honest model card
- ☐ Frontend performance: columnar data, <100ms filtering, zero dependencies

✓ Before You Walk In

- ☐ Run the entire pipeline once locally to verify it works: `python analytics/scripts/run_all.py`
- ☐ Check the dashboard loads and filters work: open `frontend/dashboard/index.html`

- ☐ Verify the KPI numbers in `reports/kpi_summary.json` (net profit \$220,183.99, Sharpe 0.799, etc.)
- ☐ Review `reports/business_insights.md` and `reports/data_quality_report.md` so you can speak to findings with authority
- ☐ Prepare one concrete example of each (schema design choice, SQL pattern, ML guardrail, metric definition)
- ☐ Be ready to whiteboard: data flow diagram, star-schema sketch, one SQL query with window functions
- ☐ Have a backup answer ready for "if you had more time, what would you add?" (walk-forward validation, dbt, Postgres, live dashboards)